

Intoduction To Programming Using C++ Level 2

Dereferencing Pointer



Introduction To Programming Using C++

Level 2

Dereferencing Pointer

Definition

Dereferencing a Pointer

 Dereferencing means using the `*` operator with a pointer to access the value stored at the memory address held by that pointer.

 If `p` stores the address of `a`, then `*p` gives us the value stored in `a`.

Explanation

Pointer

-  A pointer stores the memory address of another variable.

-  For example, if $p = \&a$, then p contains the address of a .

Dereferencing

-  When we use $*p$, we access the value stored at the address contained in p .

-  If a contains 10, then $*p$ is also 10.

-  We can also use $*p$ to modify the original variable.

Procedures

1. Create a variable and assign it a value.
2. Create a pointer of the same type.
3. Store the address of the variable inside the pointer using &.
4. Print the variable's address.
5. Print the pointer's value.
6. Use *p to access the value stored at the address.
7. Use *p to modify the original variable.

💡 Example

◆ Dereferencing a Pointer

```
int main()
{
    int a = 10;
    int* p;
    p = &a;
    cout << "Address of A = " << &a << endl;
    cout << "P = " << p << endl;
    cout << "*P = " << *p << endl;
    cout << "Address of P = " << &p << endl;
    *p = 20;
    cout << "A after *p = 20: " << a << endl;
    cout << "*P after *p = 20: " << *p << endl;
    return 0;
}
```

➡ Result

Address of A = 0x...

P = 0x...

*P = 10

Address of P = 0x...

After:

*p = 20

Note

■ The value of p

▨ p contains the memory address of a.

▨ Therefore, p and &a contain the same address.

■ The value of *p

▨ *p gives the value stored at the address contained in p.

▨ If p points to a, then *p gives the value of a.

■ The address of p

▨ &p gives the memory address of the pointer variable itself.

▨ This is different from the address stored inside p.

⊘ Warning

◆ Do not dereference an uninitialized or invalid pointer.

◆ For example:

```
int* p;  
*p = 20;
```

◆ This is incorrect because p does not point to a valid variable.

◆ A pointer should contain a valid address before using *p.



Comparison



p

- ✓ Contains the address of the variable.
- ✓ If p points to a, then p is equivalent to the address of a.



*p

- ✓ Accesses the value stored at the address contained in p.
- ✓ If p points to a, then *p gives the value of a.



&p

- ✓ Gives the address of the pointer variable p itself.
- ✓ This is different from p.

✦ Summary

- ✓ $p \rightarrow$ The address stored in the pointer.
- ✓ $*p \rightarrow$ The value stored at the address held by the pointer.
- ✓ $\&a \rightarrow$ The address of variable a .
- ✓ $\&p \rightarrow$ The address of the pointer variable itself.
- ✓ If p points to a , changing $*p$ changes a .
- ✓ Dereferencing allows us to access and modify the original variable through its memory address.

Thank you 😊